Pseudocode for Fourier Epicycle Reconstruction

Mustafa Bakir

2/10/2025

1. Resample Function

Description: Resamples an array of points to have n evenly spaced points along the path.

Input:

- `points[]`: Array of points (e.g. p5.js vectors).
- n: Number of points to resample to.

Output:

- `newPoints[]`: Resampled array of n evenly spaced points.

Algorithm 1: Resample(points[], n)

Function `Resample(`*points[], n*`):`
- **if** *length(points)* < 2 **then**
  - **return** *points* `// Fewer than 2 points, return original array`
- totalLength $\leftarrow 0$
- **for** $i \leftarrow 1$ ***to*** *length(points)* -1 **do**
  - totalLength $\leftarrow$ totalLength + Distance(*points*$[i-1]$, *points*$[i]$)
- interval $\leftarrow \dfrac{\text{totalLength}}{n-1}$
- newPoints $\leftarrow$ [*points*[0]]
- $d \leftarrow 0$
- **for** $i \leftarrow 1$ ***to*** *length(points)* -1 **do**
  - prev $\leftarrow$ *points*$[i-1]$
  - curr $\leftarrow$ *points*$[i]$
  - segmentDist $\leftarrow$ Distance(prev, curr)
  - **while** $d +$ *segmentDist* $\geq$ *interval* **do**
    - $t \leftarrow \dfrac{\text{interval} - d}{\text{segmentDist}}$
    - $nx \leftarrow$ Lerp(*prev.x*, *curr.x*, t)
    - $ny \leftarrow$ Lerp(*prev.y*, *curr.y*, t)
    - newPoint $\leftarrow$ CreateVector(nx, ny)
    - Append newPoint to newPoints
    - prev $\leftarrow$ newPoint
    - segmentDist $\leftarrow$ Distance(prev, curr)
    - $d \leftarrow 0$
  - $d \leftarrow d +$ segmentDist
- **return** newPoints

2. Discrete Fourier Transform (DFT) Function

Description: Computes the Discrete Fourier Transform (DFT) of an array of complex numbers, adjusting the frequency for indices greater than $N/2$.

Input:

- `x[]`: Array of complex numbers represented by objects $\{\text{re}, \text{im}\}$ (real and imaginary parts).

Output:

- `X[]`: Array of DFT coefficients with frequency, amplitude, and phase information.

Recall that the Fourier coefficient for index k is given by

$$X[k] = \frac{1}{N} \sum_{n=0}^{N-1} x[n]\, e^{-i\,\frac{2\pi k n}{N}},$$

which in real/imaginary parts becomes:

$$\text{re} = \frac{1}{N} \sum_{n=0}^{N-1} \left(x[n].\text{re} \cdot \cos\left(\frac{2\pi k n}{N}\right) + x[n].\text{im} \cdot \sin\left(\frac{2\pi k n}{N}\right)\right),$$
$$\text{im} = \frac{1}{N} \sum_{n=0}^{N-1} \left(-x[n].\text{re} \cdot \sin\left(\frac{2\pi k n}{N}\right) + x[n].\text{im} \cdot \cos\left(\frac{2\pi k n}{N}\right)\right).$$

Algorithm 2: DFT(x[])

Function `DFT(`*x[]*`)`**:**
- $N \leftarrow \text{length}(x)$
- Initialize $X[] \leftarrow \{\}$
- **for** $k \leftarrow 0$ ***to*** $N-1$ **do**
  - $re \leftarrow 0$
  - $im \leftarrow 0$
  - **for** $n \leftarrow 0$ ***to*** $N-1$ **do**
    - $\phi \leftarrow \dfrac{2\pi\, k\, n}{N}$
    - $re \leftarrow re + x[n].\text{re} \cdot \cos(\phi) + x[n].\text{im} \cdot \sin(\phi)$
    - $im \leftarrow im - x[n].\text{re} \cdot \sin(\phi) + x[n].\text{im} \cdot \cos(\phi)$
  - $re \leftarrow \dfrac{re}{N}$
  - $im \leftarrow \dfrac{im}{N}$
  - $amp \leftarrow \sqrt{re^2 + im^2}$
  - $phase \leftarrow \text{atan2}(im, re)$
  - $freq \leftarrow k$
  - **if** $k > \dfrac{N}{2}$ **then**
    - $freq \leftarrow k - N$
  - Append $\{re, im, freq, amp, phase\}$ to $X[]$
- **return** $X[]$

3. Epicycle Update and Display Functions

Description: Represents one epicycle for Fourier reconstruction, calculates its position at a given time, and displays the corresponding circle.

Input:

- `time`: Current time.
- `coeff`: Fourier coefficient containing freq, amp, and phase.

Output:

- `nextPosition`: New position of the epicycle based on the time.

Algorithm 3: Epicycle Class

Class *Epicycle*:

`Constructor(`*Epicycle(*x, y, $coeff$*)*`)` this.x $\leftarrow x$ `// Center` x`-coordinate`
this.y $\leftarrow y$ `// Center` y`-coordinate`
this.freq $\leftarrow coeff.freq$ `// Frequency from coefficient`
this.amp $\leftarrow coeff.amp \times$ scaleFactor `// Amplitude with scaling`
this.phase $\leftarrow coeff.phase$ `// Phase from coefficient`
`Update(`*Update(time)*`)` $angle \leftarrow$ this.phase $+ 2\pi \times$ this.freq $\times time$
$dx \leftarrow$ this.amp $\times \cos(angle)$
$dy \leftarrow$ this.amp $\times \sin(angle)$
nextPosition $\leftarrow$ CreateVector(this.x $+ dx$, this.y $+ dy$)
return nextPosition
`Display(`*Display()*`)` SetStroke(255, 100) `// Set stroke color with transparency`
NoFill()
DrawCircle(this.x, this.y, $2 \times$ this.amp) `// Draw epicycle circle (diameter` $= 2 \times$ `amp)`

4. Main Loop for Drawing Fourier Reconstruction

Description: Iterates through the Fourier coefficients, draws the epicycles, and reconstructs the shape over time. Updates the drawing path and the time variable.

Input:

- `time`: Time variable incremented each frame.
- `numEpicycles`: Number of epicycles to use for reconstruction.

Output: Draws the Fourier reconstruction path on the canvas.

Function `Draw()`:

```
ClearCanvas(0)                                              // Clear the canvas with black
if drawingDone then
    prev ← CreateVector(width/2, height/2)                  // Start at canvas center
    for i ← 0 to numEpicycles − 1 do
        epi ← CreateEpicycle(prev.x, prev.y, fourier[i])
        epi.Display()
        next ← epi.Update(time)
        SetStroke(255)                                      // White stroke
        DrawLine(prev.x, prev.y, next.x, next.y)
        prev ← next
    Append prev to path[]
    if length(path) > 1000 then
        RemoveLast(path)                                    // Remove the oldest point in the path
    SetStroke(255, 0, 255)                                  // Magenta for reconstruction path
    NoFill()
    BeginShape()
    for each point in path do
        AddVertex(point.x, point.y)
    EndShape()

    time ← time + dt
    if time > 2π then
        time ← 0
        path ← []                                           // Clear the reconstruction path
```